

Ingeniería de Software II

2026 – CLASE 7

Contenidos

❖ Gestión de Proyectos

- Métricas
- Estimaciones

Gestión de Proyectos

Métricas

- ❖ Las métricas son fundamentales para gestionar y mejorar procesos o sistemas.
- ❖ Se apoyan en cuatro objetivos principales:
 - Entender
 - Controlar
 - Mejorar
 - Evaluar



Métricas - Definiciones

- ❖ A diferencia de una simple medida, una métrica a menudo implica un cálculo o una fórmula que relaciona varias medidas para proporcionar una comprensión más profunda.
- ❖ Suele involucrar datos organizados y analizados para aportar significado.

Métricas - Definiciones

Medida



Medición



Métrica

LOC por punto de función			
Lenguaje	LOC/RP	Lenguaje	LOC/RP
Entendador	335	Basic, ANSI/QuickTurbo	84
Macros/Entendador	213	Java	53
C	150	Visual C++	34
Fortran	108	Fortran 2.5	24
Cobol	108	Visual Basic	22
Pascal	91	Delphi	29
Cobol ANSI 85	91	C++	29
Basic	91	Visual Cobol	29
RPO	80	Clipper	19
PLI	80	Powers Builder	16
Ada	71	Visual C++	6

Indicador



Métricas - Definiciones

- ❖ Las métricas pueden ser utilizadas para que los profesionales e investigadores puedan tomar las mejores decisiones.
- ❖ Se utilizan para asegurar la calidad en:
 - Proyectos Software
 - Procesos
 - Productos

Métricas del proyecto

- ❖ Sirven para cumplir propósitos tácticos.
- ❖ Uso:
 - Ajustes en el calendario y evitar demoras
 - Valorar el estado de un proyecto en marcha
 - Rastrear riesgos
 - Descubrir áreas de problemas
 - Ajustar flujo de trabajo/tareas
 - Evaluar habilidad del equipo



Métricas del proceso

❖ Propósitos estratégicos



Recopilación

a través de todos los proyectos y por un espacio de tiempo.



Intención

proporcionar un conjunto de indicadores para mejorar el proceso.

Métricas del proceso

❖ Uso:

- Sentido común y sensibilidad organizacional
- Retroalimentación
- No usar métricas para valorar a los individuos
- Establecer metas y métricas claras
- No excluir métricas



Métricas del producto

❖ Dinámicas

- Hechas en un programa en ejecución.
- Ayudan a valorar la eficiencia y fiabilidad de un programa.



Se relacionan directamente con los atributos de calidad del software

¿Cómo las medimos?

❖ Estáticas

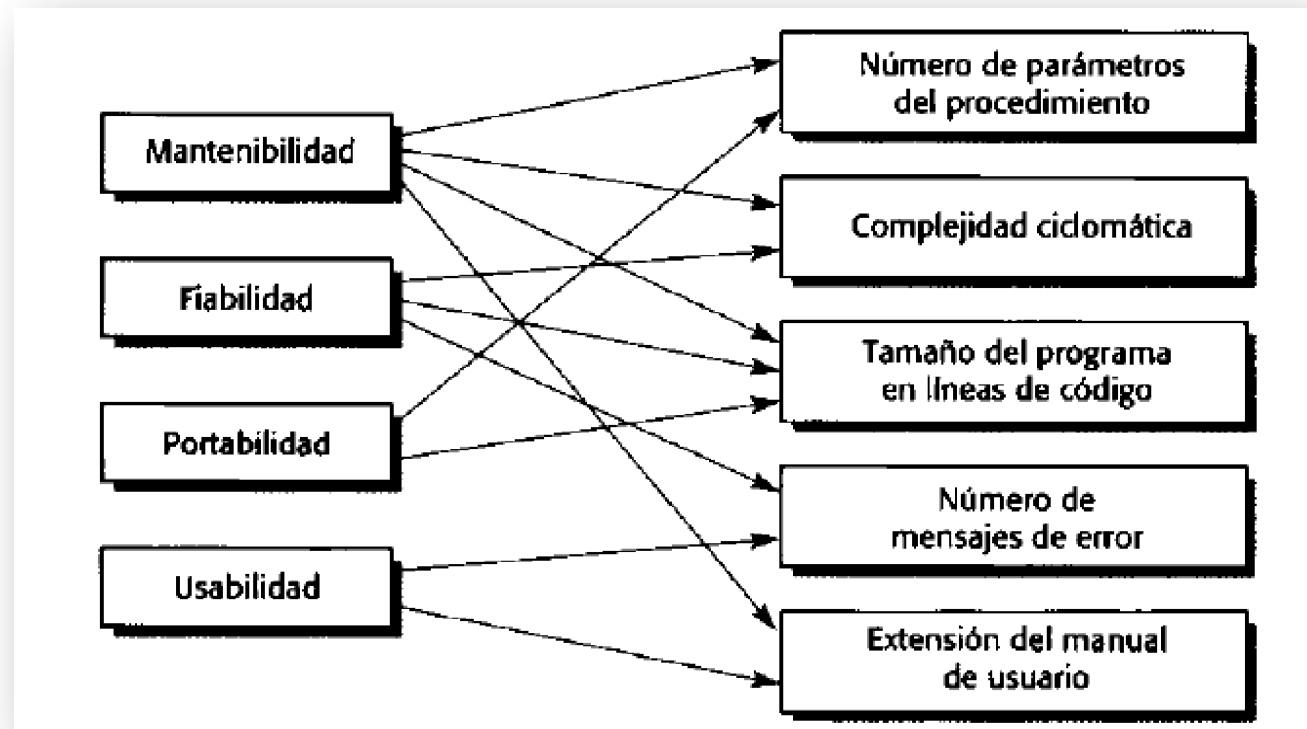
- Hechas de representaciones del sistema.
- Ayudan a valorar la complejidad, comprensibilidad y mantenibilidad.

```
rule_id = $role_details['id'],
resource_id = $resource_details['id'],
);
if ( $access == false ) {
    // Remove the rule as there is currently no need for it
    $details['access'] = $access;
    $this->sql->delete( 'acl_rules', $details );
} else {
    // Update the rule with the new access value
    $this->sql->update( 'acl_rules', array( 'access' => $access ) );
}
foreach( $this->rules as $key=>$rule ) {
    if ( $details['role_id'] == $rule['role_id'] && $details['resource_id'] == $rule['resource_id'] ) {
        if ( $access == false ) {
            unset( $this->rules[ $key ] );
        } else {
            // Update the rule with the new access value
            $this->sql->update( 'acl_rules', array( 'access' => $access ) );
        }
    }
}
```

Se relacionan, de manera indirecta, con los atributos de calidad del software

Métricas del producto

❖ Atributos de calidad externos vs Atributos internos



Métricas estáticas del producto

Fan-in/Fan-out	Fan-in es una medida del número de funciones o métodos que llaman a otra función o método (por ejemplo, X). Fan-out es el número de funciones que son llamadas por una función X. Un valor alto de fan significa que X está fuertemente acoplada al resto del diseño y que los cambios en X tendrán muchos efectos importantes. Un valor alto de fan-out sugiere que la complejidad de X podría ser alta debido a la complejidad de la lógica de control necesaria para coordinar los componentes llamados.
Longitud del código	Ésta es una medida del tamaño del programa. Generalmente, cuanto más grande sea el tamaño del código de un componente, más complejo y susceptible de errores será el componente. La longitud del código ha mostrado ser la métrica más fiable para predecir errores en los componentes.
Complejidad ciclomática	Ésta es una medida de la complejidad del control de un programa. Esta complejidad del control está relacionada con la comprensión del programa. El cálculo de la complejidad ciclomática se trata en el Capítulo 22.
Longitud de los identificadores	Es una medida de la longitud promedio de los diferentes identificadores en un programa. Cuanto más grande sea la longitud de los identificadores, más probable será que tengan significado; por lo tanto, el programa será más comprensible.
Profundidad del anidamiento de las condicionales	Ésta es una medida de la profundidad de anidamiento de las instrucciones condicionales «if» en un programa. Muchas condiciones anidadas son difíciles de comprender y son potencialmente susceptibles de errores.
Índice de Fog	Ésta es una medida de la longitud promedio de las palabras y las frases en los documentos. Cuanto más grande sea el Índice de Fog, el documento será más difícil de comprender.

Métricas estáticas del producto

Métrica orientada a objetos	Descripción
Métodos ponderados por clase (<i>weighted methods per class</i> , WMC)	Éste es el número de métodos en cada clase, ponderado por la complejidad de cada método. Por lo tanto, un método simple puede tener una complejidad de 1, y un método grande y complejo tendrá un valor mucho mayor. Cuanto más grande sea el valor para esta métrica, más compleja será la clase de objeto. Es más probable que los objetos complejos sean más difíciles de entender. Tal vez no sean lógicamente cohesivos, por lo que no pueden reutilizarse de manera efectiva como superclases en un árbol de herencia.
Profundidad de árbol de herencia (<i>depth of inheritance tree</i> , DIT)	Esto representa el número de niveles discretos en el árbol de herencia en que las subclases heredan atributos y operaciones (métodos) de las superclases. Cuanto más profundo sea el árbol de herencia, más complejo será el diseño. Es posible que tengan que comprenderse muchas clases de objetos para entender las clases de objetos en las hojas del árbol.
Número de hijos (<i>number of children</i> , NOC)	Ésta es una medida del número de subclases inmediatas en una clase. Mide la amplitud de una jerarquía de clase, mientras que DIT mide su profundidad. Un valor alto de NOC puede indicar mayor reutilización. Podría significar que debe realizarse más esfuerzo para validar las clases base, debido al número de subclases que dependen de ellas.

Acoplamiento entre clases de objetos (<i>coupling between object classes</i> , CBO)	Las clases están acopladas cuando los métodos en una clase usan los métodos o variables de instancia definidos en una clase diferente. CBO es una medida de cuánto acoplamiento existe. Un valor alto para CBO significa que las clases son estrechamente dependientes y, por lo tanto, es más probable que el hecho de cambiar una clase afecte a otras clases en el programa.
Respuesta por clase (<i>response for a class</i> , RFC)	RFC es una medida del número de métodos que potencialmente podrían ejecutarse en respuesta a un mensaje recibido por un objeto de dicha clase. Nuevamente, RFC se relaciona con la complejidad. Cuanto más alto sea el valor para RFC, más compleja será una clase y, por ende, es más probable que incluya errores.
Falta de cohesión en métodos (<i>lack of cohesion in methods</i> , LCOM)	LCOM se calcula al considerar pares de métodos en una clase. LCOM es la diferencia entre el número de pares de método sin compartir atributos y el número de pares de método con atributos compartidos. El valor de esta métrica se debate ampliamente y existe en muchas variaciones. No es claro si realmente agrega alguna información útil además de la proporcionada por otras métricas.

Métricas del producto

❖ LDC – Líneas de código

- » Es la métrica más común para medir el tamaño de un producto
- » Es una medida discutida 
- » Permite obtener resultados:
 - » Tiempo estimado 
 - » Cantidad de desarrolladores necesarios 
- » Si una organización de software mantiene registros sencillos, se puede crear una tabla de datos orientados al tamaño
- » Se realiza post mortem

Métricas del producto

- ❖ Utilidad de las métricas post mortem:
 - » Conformar una línea base para futuras métricas.
 - » Ayudar al mantenimiento conociendo la complejidad lógica, tamaño, flujo de información, identificando módulos críticos.
 - » Ayudar en los procesos de reingeniería.



Métricas del producto

- ❖ Métricas orientadas al tamaño, como **KLDC** (miles de líneas de código), permiten obtener:
 - » *Productividad*: relación entre KLDC / Persona mes
 - » *Calidad*: relación entre Errores / KLDC
 - » *Costo*: relación entre \$ / KLDC

¿Cómo manejo las líneas en blanco, comentarios, etc.?

Propuesta Fenton/Pfleeger

- Medir: CLOC = Cantidad de líneas de comentarios
- Luego: long total (LOC) = NCLOC + CLOC
- Surgen medidas indirectas: CLOC/LOC mide la densidad de comentarios

Métricas del producto - Ejemplo

- ❖ Calcular, usando LDC, la productividad, calidad y costo para los cuatro proyectos de los cuales se proporcionan los datos:

Proyecto	LDC	U\$S	Errores	Personas-mes	Errores/KLDC	U\$S/KLDC	KLDC/Personas-mes
P1	25.500	15000	567	15	22,23	588,23	1,7
P2	19.100	7200	210	10	10,99	376,96	1,91
P3	10.700	6000	100	20	9,34	560,74	0,53
P4	100.000	18000	2200	30	22	180	3,33

- » *Productividad:* KLDC / Persona mes
- » *Calidad:* errores/ KLDC
- » *Documentación:* págs. Doc./ KLDC
- » *Costo:* \$ / KLDC

- ¿Cuál es el proyecto de **mayor calidad** (errores/KLDC)?
- ¿Cuál es el proyecto de **mayor costo por línea** (\$/KLDC)?
- ¿Cuál es el proyecto de **menor productividad por persona** (KLDC/personas-mes)?

Métricas del producto

- ❖ Métricas de funcionalidad de un sistema, como **Punto Función (PF)**, permiten obtener:
 - » *Productividad*: relación entre PF / Persona mes
 - » *Calidad*: relación entre Errores / PF
 - » *Costo*: relación entre \$ / PF
- ❖ Es una Métrica temprana

Medida subjetiva independiente del lenguaje, de estimación más fácil

Métricas del producto

- ❖ PF – Punto Función, mide cuánta funcionalidad ofrece un sistema según su especificación, no cuántas líneas de código tiene.

Suma de todos los elementos ponderados

$$PF = \text{TOTAL} * [0.65 + 0.01 * \sum_{i=1 \text{ a } 14} (F_i)]$$

Factores de ajuste (14 preguntas sobre el sistema) $0 \leq F_i \leq 5$

Tipos de elementos		simple	medio	complejo	
# Entradas	*	[3 4 6]	=		
# Salidas	*	[4 5 7]	=		
# Consultas	*	[3 4 6]	=		
# Almacenamientos internos	*	[7 10 15]	=		
# Interfaces externas	*	[5 7 10]	=		
					TOTAL

El factor de ponderación es subjetivo, y está dado por la organización/equipo

Métricas del producto

❖ Factores de ajuste de Punto Fu

» Son 14 preguntas que evalúan sistema.

» Cada una de las preguntas se puntúa

0 = no influye

5 = influye mucho

0	1	2
No influencia	Incidental	Moderado

1. **Comunicación de datos** → ¿el sistema intercambia datos con otros?
2. **Procesamiento distribuido** → ¿corre en varios servidores/nodos?
3. **Rendimiento** → ¿hay requisitos estrictos de velocidad?
4. **Configuración del sistema** → ¿el hardware/infraestructura es especial?
5. **Volumen de transacciones** → ¿maneja muchas operaciones?
6. **Entrada de datos online** → ¿los usuarios ingresan datos en tiempo real?
7. **Eficiencia del usuario final** → ¿la interfaz debe ser muy optimizada?
8. **Actualización online** → ¿los datos se actualizan en tiempo real?
9. **Procesamiento complejo** → ¿hay lógica complicada?
10. **Reusabilidad** → ¿el sistema debe poder reutilizarse?
11. **Facilidad de instalación** → ¿debe ser fácil de instalar?
12. **Facilidad de operación** → ¿debe ser fácil de usar/administrar?
13. **Múltiples sitios** → ¿se usa en varias ubicaciones?
14. **Facilidad de cambio** → ¿debe adaptarse fácilmente a cambios?

Desarrollo de una métrica

- ❖ Método **GQM** (Goal, Question, Metric)
 - » Desarrollada por Victor Basili.
 - » Orientado a lograr una métrica que “mida” cierto objetivo, el cual nos permita mejorar la calidad de nuestro proyecto.



<https://www.cs.umd.edu/~basili/publications/technical/T78.pdf>

Desarrollo de una métrica

❖ Método GQM - Estructura

Objetivo

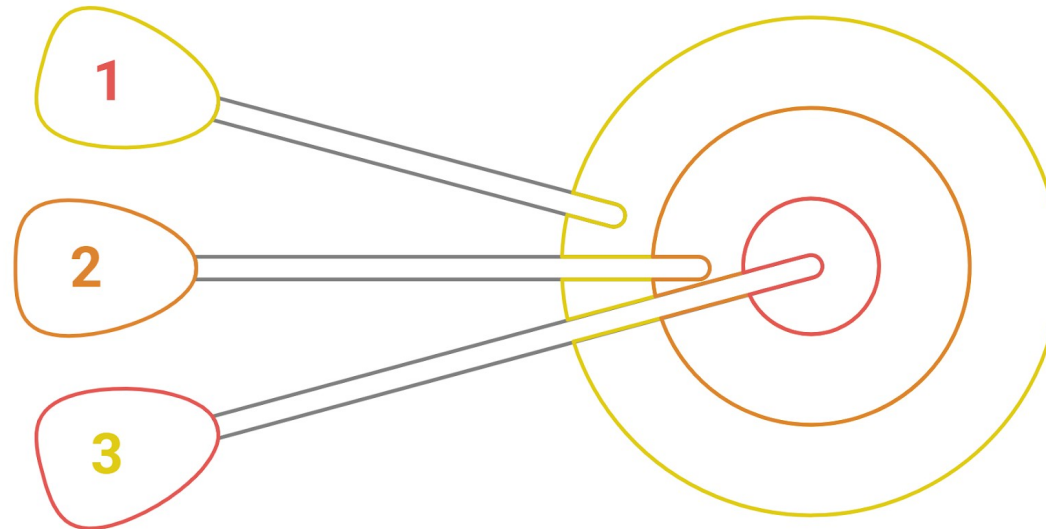
El objetivo central del proyecto

Preguntas

Preguntas para verificar el cumplimiento del objetivo

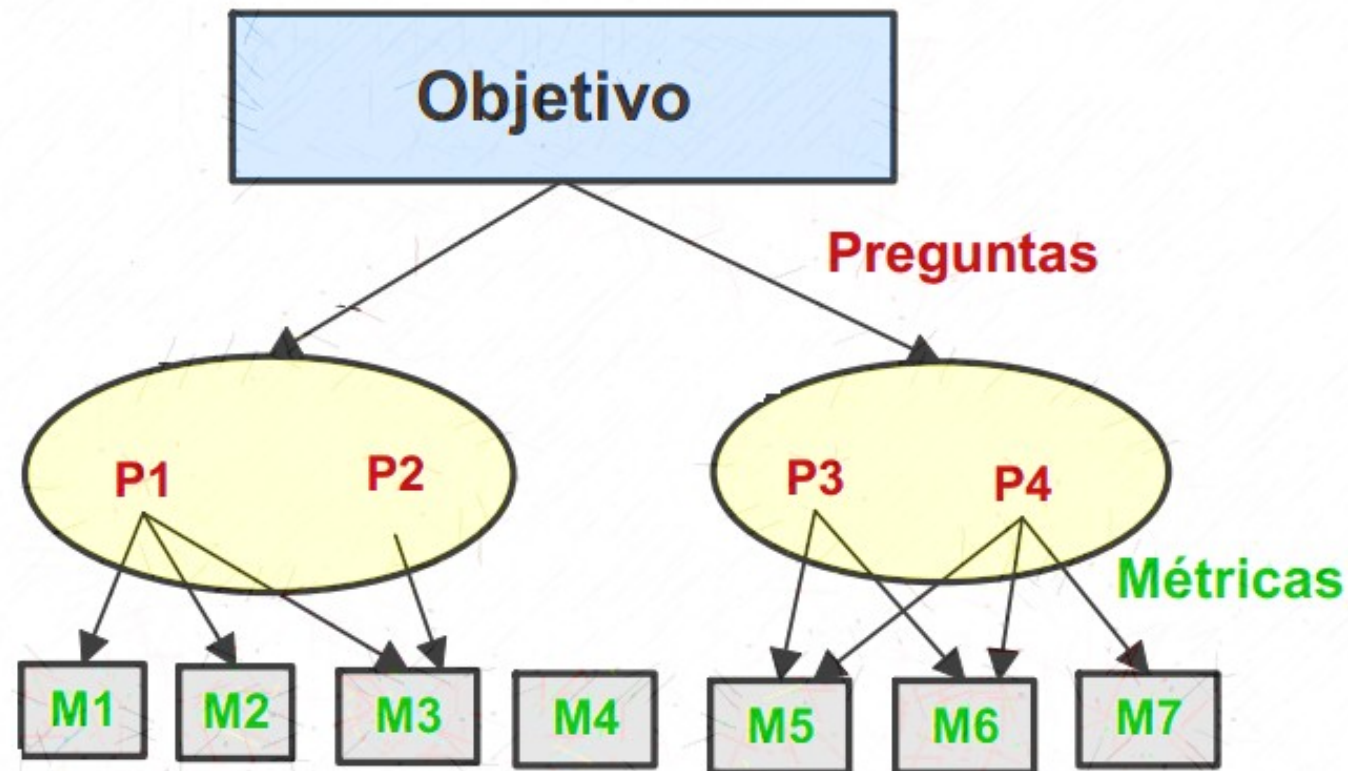
Métricas

Medidas cuantitativas para responder a las preguntas



Desarrollo de una métrica

❖ Método GQM



Desarrollo de una métrica – Ejemplo GQM

- ❖ Evaluamos, en la etapa de Análisis de Requerimientos, la tarea Asignación de responsabilidades.

Propósito	Evaluar	
Característica	Asignación de responsabilidades	
Punto de Vista	Gerencia de Proyecto	
Pregunta 1	¿Existe un proceso para la asignación de roles?	
	M1	Valor Binario
Pregunta 2	¿Hay un responsable de asignar roles?	
	M2	Valor Binario
Pregunta 3	¿El responsable siempre realiza su tarea?	
	M3	Valor Binario
Pregunta 4	¿Existe información anterior sobre las tareas realizadas por cada integrante?	
	M4	Valor Binario
Pregunta 5	¿Esa información está disponible?	
	M5	Valor Binario

Desarrollo de una métrica – Ejemplo GQM

❖ Indicadores:

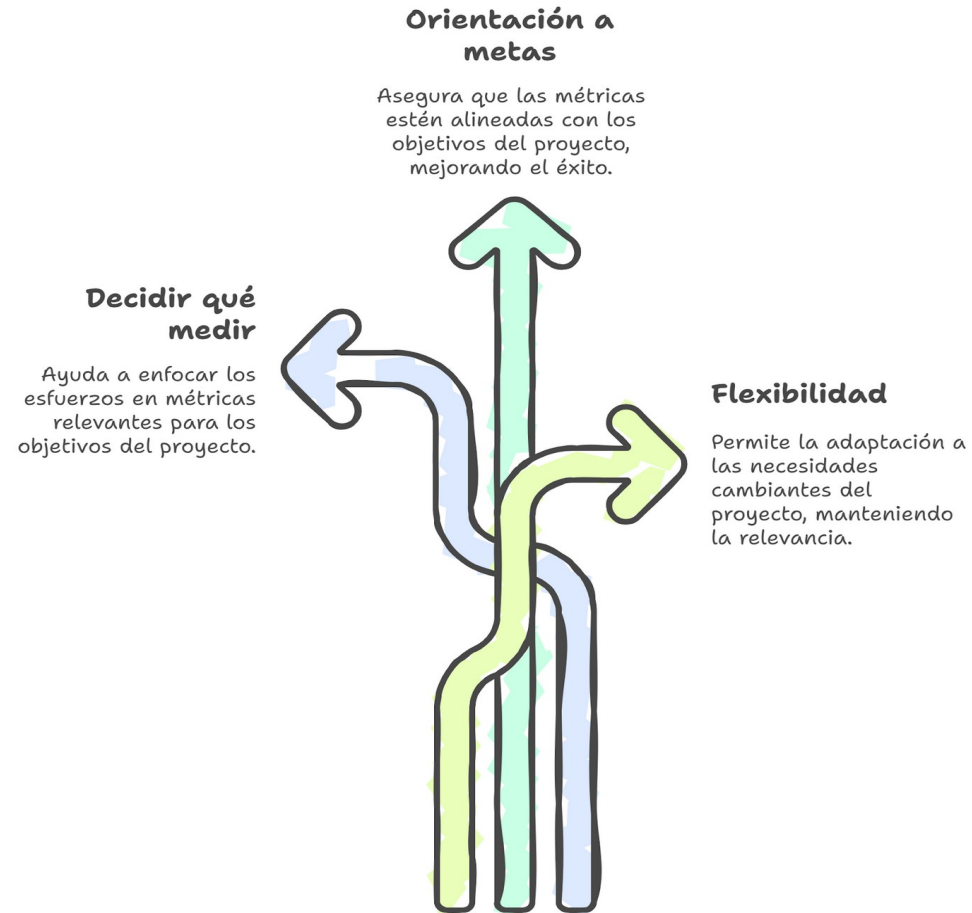
<u>Nombre</u>	<u>Descripción</u>	<u>Fórmula</u>
I1	Gestión de Asignación de roles	M2 & M3 & M4 & M5
I2	Proceso de Asignación de roles	M1 & M4 & M5

- ❖ A partir de los indicadores definidos, se propone realizar el control de la meta a través de un tablero de control de indicadores específicos. Podemos decir que nuestra meta se cumple si los indicadores muestran los siguientes valores:

I1	Gestión de Asignación de roles	Verdadero
I2	Proceso de Asignación de roles	Verdadero

Desarrollo de una métrica

❖ Método GQM



Gestión de Proyectos

Estimaciones

- ❖ La estimación del software es el proceso de predecir la cantidad más realista de esfuerzo (expresado comúnmente en horas persona o costo monetario) requerido para desarrollar o mantener software.



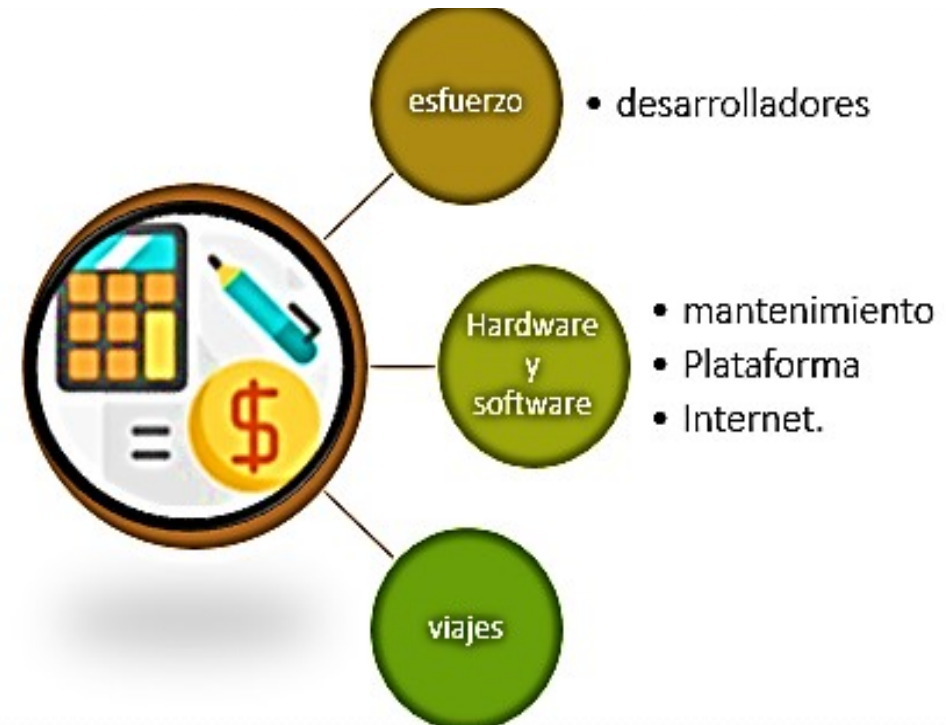
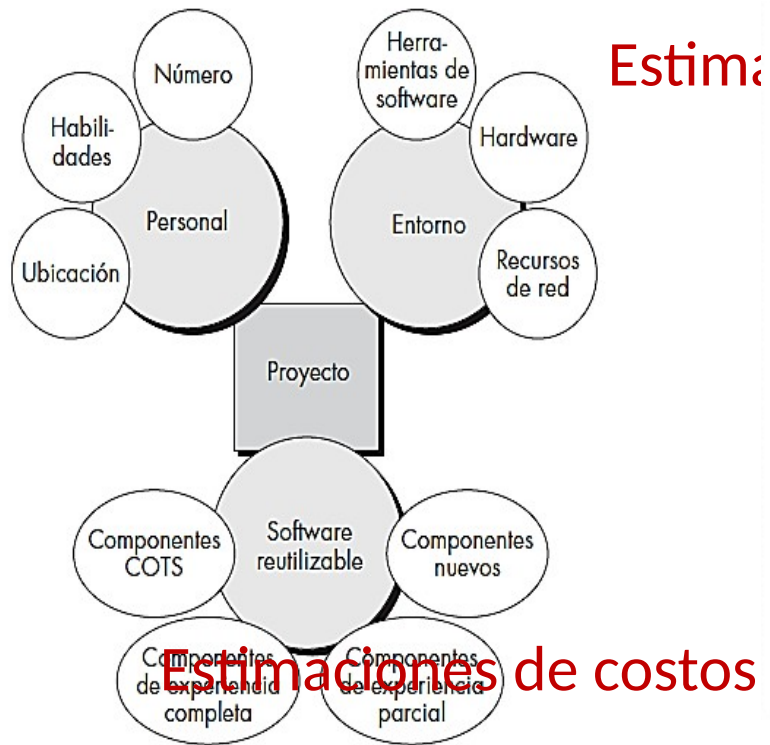
Estimaciones y métricas

- ❖ Las métricas y la estimación de software se enfocan en la perspectiva temporal.



Estimaciones

❖ ¿Qué podemos estimar?



Estimaciones

❖ ¿Qué tener en cuenta?

Oportunidad de mercado	Una organización de desarrollo podría ofertar un bajo precio debido a que desea conseguir cuota de mercado. Aceptar un beneficio bajo en un proyecto podría darle la oportunidad de obtener más beneficios posteriormente. La experiencia obtenida le permite desarrollar nuevos productos.
Incertidumbre en la estimación de costes	Si una organización está insegura de su coste estimado, puede incrementar su precio por encima del beneficio normal para cubrir alguna contingencia.
Términos contractuales	Un cliente puede estar dispuesto a permitir que el desarrollador retenga la propiedad del código fuente y que reutilice el código en otros proyectos. Por lo tanto, el precio podría ser menor que si el código fuente del software se le entregara al cliente.
Volatilidad de los requerimientos	Si es probable que los requerimientos cambien, una organización puede reducir los precios para ganar un contrato. Después de que el contrato se le asigne, se cargan precios altos a los cambios en los requerimientos.
Salud financiera	Los desarrolladores en dificultades financieras podrían bajar sus precios para obtener un contrato. Es mejor tener beneficios más bajos de los normales o incluso quebrar antes de quedar fuera de los negocios.

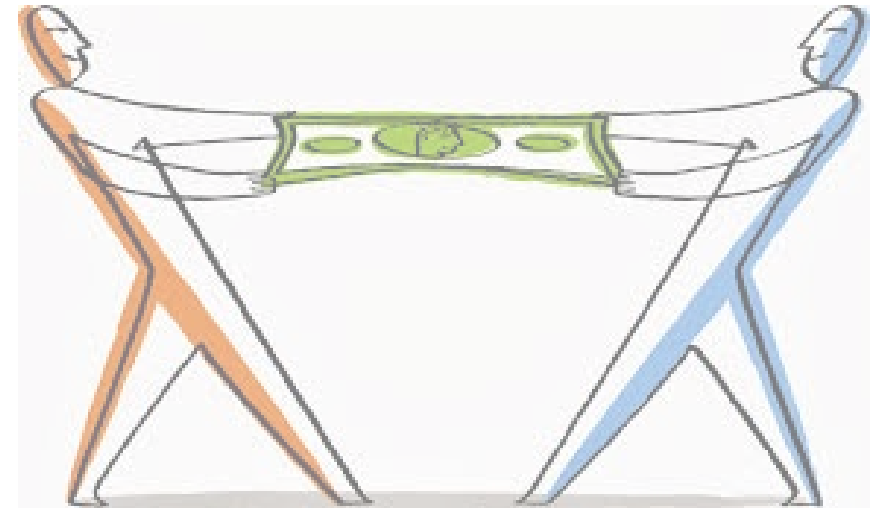
Estimaciones

❖ Fijación de precio

» Debe pensarse en:

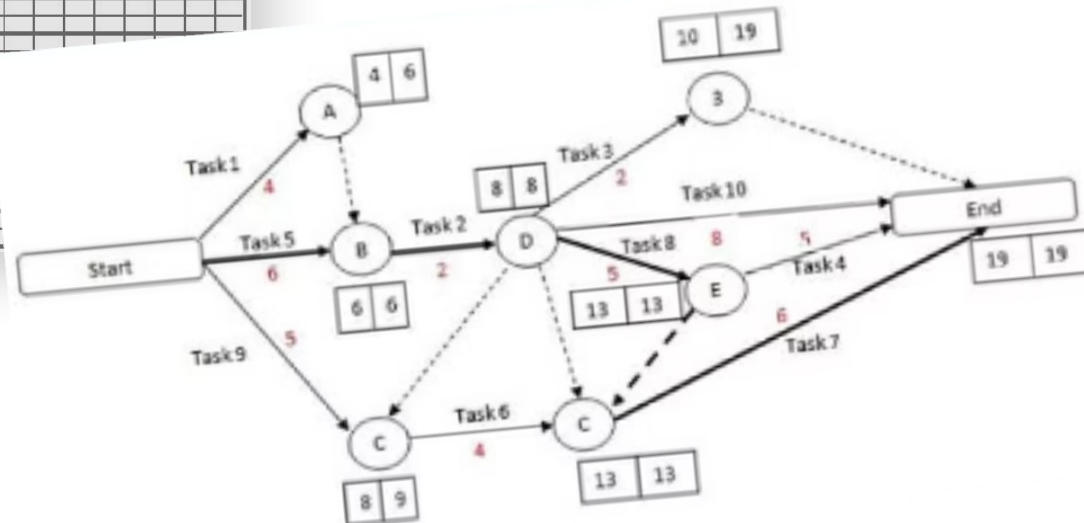
- los intereses de la empresa
- los riesgos
- el tipo de contrato

» Esto puede hacer que el precio suba o baje.



Estimaciones de tiempo

Carta Gantt (Control de las Actividades Proyecto Servicio)																									
Nombre del Equipo: Los Triunfadores		Curso: 2º Medio "X"												Año: 2008											
Nº	Actividades	Agosto				Septiembre				Octubre				Noviembre				Diciembre							
	Meses - Semanas	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4				
1	Redacción del Proyecto																								
2	Diseño del Servicio																								
3	Cotización de Materiales																								
4	Compra de materiales																								
5	Presentación del Proyecto																								
6	Promoción del Servicio																								
7	Prestación del Servicio																								
8	Evaluación de Proceso																								
9	Control de Calidad																								



Técnicas de estimación

Juicio experto

Consulta a varios expertos.
Estiman.
Comparan.
Discuten.

Técnica Delphi

Sucesivas rondas.
Anónimas.
Consenso.

División de Trabajo

Jerárquica hacia arriba.

Planning Poker

Basada en consenso.
Incluye todo el equipo de desarrollo.
Iterativa.

Técnicas de estimación

❖ Juicio experto

- » Se basa en la opinión y el conocimiento acumulado de un individuo o grupos que poseen experiencia y pericia relevante en el desarrollo de software, tecnologías específicas, dominios de negocio o gestión de proyectos.
- » Consiste en solicitar a una o varias personas consideradas "**expertas**" que proporcionen una estimación del esfuerzo, tiempo, costo o complejidad necesaria para completar una tarea, característica, módulo o proyecto de software determinado. Los expertos en general no son anónimos.

Técnicas de estimación

❖ Juicio experto



Técnicas de estimación

❖ Técnica Delphi

- » Método estructurado de comunicación y pronóstico que se basa en la agregación de opiniones de un panel de expertos (más de uno siempre) para llegar a un consenso sobre una estimación.
- » Se caracteriza por:
 - Anonimato: las identidades de los expertos que participan se mantienen anónimas. Esto reduce la influencia de personalidades dominantes y permite que cada experto exprese sus opiniones libremente.
 - Iteración: el proceso se lleva a cabo en múltiples rondas. En cada ronda, los expertos proporcionan sus estimaciones iniciales y las justificaciones de la estimación.
 - Retroalimentación controlada: después de cada ronda, un facilitador recopila, resume y distribuye las respuestas a todos los expertos.
 - Búsqueda de consenso: los expertos revisan y ajustan sus estimaciones en las rondas siguientes. El proceso continúa hasta alcanzar un consenso razonable de estimación.

Técnicas de estimación

❖ División de trabajo (top-down)

- » Consiste en descomponer un proyecto de software grande y complejo en componentes o actividades más pequeños, manejables y, por lo tanto, más fáciles de estimar de manera individual.
- » En esencia, el proceso implica:
 1. Descomposición: el proyecto de software se divide jerárquicamente en elementos de trabajo progresivamente más pequeños. La descomposición puede basarse en las funciones del software, las actividades del proceso de desarrollo (como análisis, diseño, codificación, pruebas) o una combinación de ambos.
 2. Estimación de las Partes: se estima el esfuerzo, el costo y/o la duración requerida para cada una de estas partes individuales. La estimación de estas partes más pequeñas tiende a ser más precisa que intentar estimar el proyecto completo de una sola vez, ya que la complejidad se reduce y es más fácil visualizar el trabajo involucrado.
 3. Composición (Agregación): las estimaciones de las partes individuales se suman o combinan para obtener una estimación global del proyecto completo.

Técnicas de estimación

❖ Planning Poker

- » Es una técnica de estimación colaborativa y basada en consenso en el desarrollo de software ágil, especialmente en los frameworks Scrum.
- » Su objetivo es obtener estimaciones más precisas del tamaño de las historias de usuario o elementos del backlog a través de la participación y el entendimiento compartido del equipo.
- » Características:
 - Participan todas las personas comprometidas en el desarrollo.
 - Se hace sobre cada una de las historias de usuario de un Sprint.
 - Se llama así porque se utilizan cartas numeradas (serie de Fibonacci).



Técnicas de estimación

❖ Planning Poker - Pasos



<https://planningpokeronline.com/>

Técnicas de estimación

❖ Planning Poker - Ejemplo

Historia de usuario: Como administrador quiero generar informes mensuales sobre la actividad de los usuarios y el rendimiento de las ventas.

Estimación: 8 puntos de historia (secuencia de Fibonacci).

Justificación: El equipo considera que esta historia de usuario es una tarea compleja. Requiere diseñar e implementar funcionalidades de informes, recopilar y agregar datos de múltiples fuentes y generar información valiosa. La complejidad y el esfuerzo que esto implica resultan en una estimación más alta.

Técnicas de estimación

❖ ¿Cuándo usar cada técnica?

Juicio experto

- Primeras etapas del proyecto, cuando la información es limitada o el alcance es aún vago.
- Proyectos pequeños o de corta duración donde la inversión en técnicas más elaboradas no se justifica.
- Si se necesita una estimación rápida y la precisión extrema no es crítica.
- Tareas o proyectos altamente especializados donde pocas personas poseen el conocimiento necesario.
- Como punto de partida para refinar con otras técnicas más adelante.

Técnica Delphi

- Si se busca obtener un consenso de un grupo diverso de expertos, incluso si están geográficamente dispersos.
- Proyectos complejos o de alto riesgo donde el sesgo de una sola persona o las dinámicas grupales pueden influir negativamente.
- Si no hay datos históricos suficientes y se depende en gran medida de la experiencia subjetiva.
- Para mitigar el efecto de la personalidad dominante en las discusiones grupales, ya que las respuestas son anónimas.

División de Trabajo

- Proyectos grandes y complejos que necesitan ser descompuestos en componentes manejables.
- Si se requiere un alto nivel de detalle y visibilidad sobre el alcance del proyecto.
- Para asignar responsabilidades claras a equipos o individuos para partes específicas del proyecto.
- Como base para la planificación, seguimiento y control del proyecto.
- Se suele usar junto con otras técnicas una vez que las tareas de bajo nivel han sido definidas.

Planning Poker

- Si se usan metodologías ágiles, para estimar HU o elementos del backlog.
- Si se desea fomentar la colaboración y el consenso dentro del equipo.
- Para identificar y discutir suposiciones, dependencias y riesgos de las tareas de forma proactiva.
- Si se busca que el equipo sea dueño de sus propias estimaciones (mayor precisión y compromiso).
- Elementos que varían en complejidad y requieren una discusión abierta para ser bien entendidos.

Modelos empíricos de estimación

❖ **COCOMO** (Constructive Cost Model)

- » Modelo de estimación de costos para proyectos de software, desarrollado por Barry Boehm.
- » Utiliza fórmulas matemáticas para predecir el esfuerzo, el tiempo y el costo de un proyecto de software en función de su tamaño, complejidad y otros factores.
- » Características:
 - Se basa en la experiencia y en datos de proyectos reales.
 - Considera factores como el tamaño del proyecto, la complejidad del código, la experiencia del equipo y el entorno de desarrollo.
 - Ofrece diferentes niveles de detalle para la estimación, desde modelos básicos hasta modelos detallados.

<https://strs.grc.nasa.gov/repository/forms/cocomo-calculation/>

Modelos empíricos de estimación

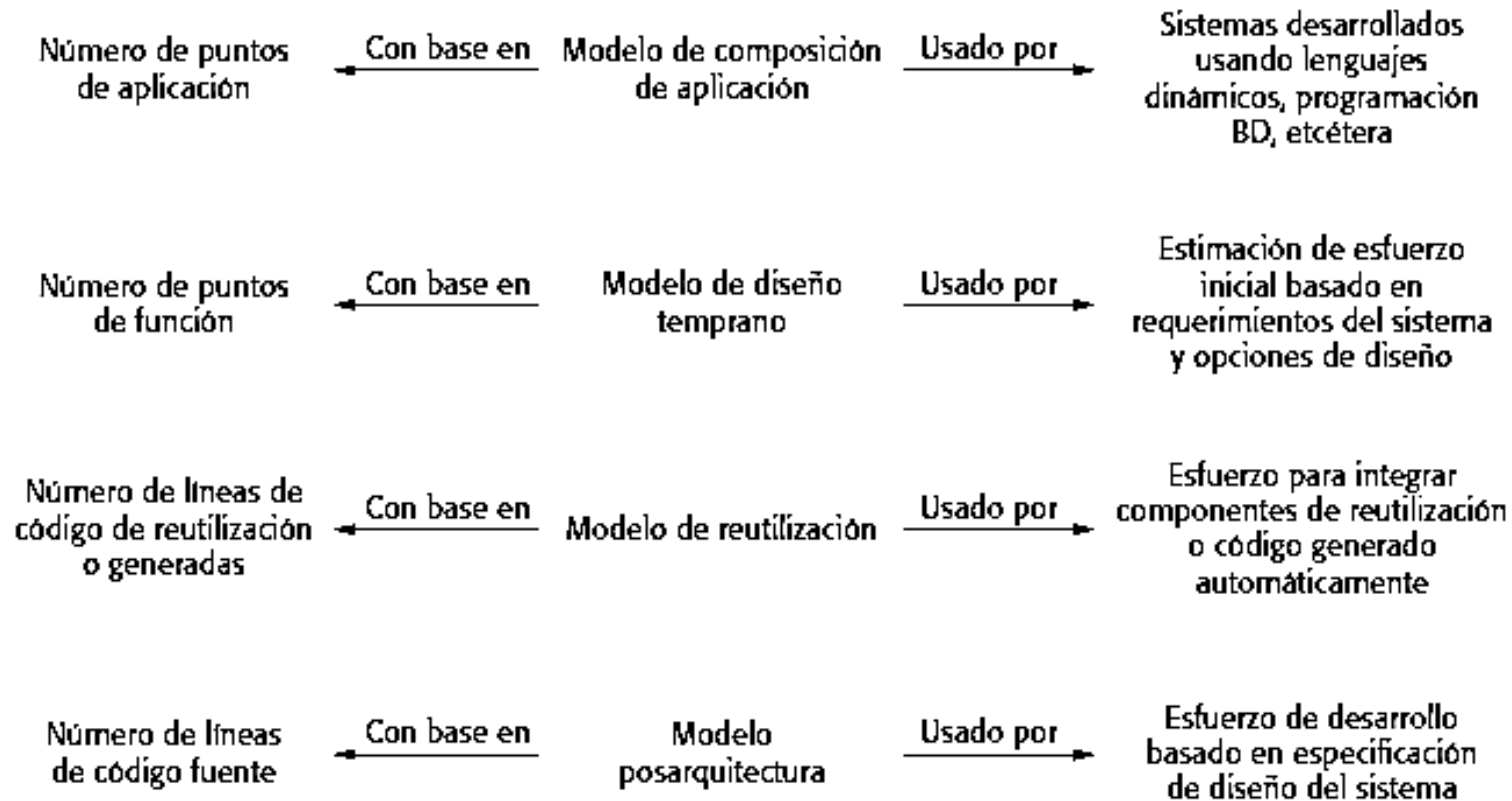
❖ COCOMO II

- » Modelo empírico que derivó de recopilar datos a partir de un gran número de proyectos de software.
- » Utilizan fórmulas derivadas empíricamente para predecir costos o esfuerzo requerido en el proyecto.
- » Vinculan:
 - el tamaño del sistema y los factores del producto, proyecto y equipo
 - el esfuerzo para desarrollar el sistema

<http://softwarecost.org/tools/COCOMO/>

Modelos empíricos de estimación

❖ COCOMO II - Modelos



Modelos empíricos de estimación

❖ COCOMO II – Modelo de composición de aplicación

- » Modela el *esfuerzo* requerido para desarrollar sistemas creados a partir de componentes de reutilización, y proyectos de creación de prototipos.
- » Se basa en una *estimación* de puntos de aplicación (o puntos objetos).
- » El número de *puntos de aplicación* en un programa es ponderada del número de pantallas que se despliegan, el número de informes que se producen, el número de módulos de programación imperativa (como Java) y el número de lenguaje de script (*scripting language*).

$$PM = (NAP \times (1 - \%reutilización/100))/PROD$$

Parámetros

PM = esfuerzo estimado en personas/mes.

NAP = total de puntos de aplicación.

PROD = productividad medida en puntos objeto.

Modelos empíricos de estimación

❖ COCOMO II – Modelo de diseño temprano

- » Puede usarse durante las primeras etapas de un proyecto antes de que esté disponible un diseño arquitectónico detallado para el mismo.
- » Supone que se acordaron los requerimientos del usuario y se comienza a marcha las etapas iniciales del proceso de diseño del sistema.
- » La meta en esta etapa debe ser elaborar una estimación preliminar y una aproximada de los costos.

$$PM = A \times \text{Tamaño}^B \times M$$

Parámetros

PM = esfuerzo estimado en personas/mes.

A = 2,94. (según Boehm)

Tamaño = KLDC (miles de líneas de código fuente).

B = esfuerzo requerido conforme aumenta el tamaño del proyecto.

Puede variar de 1,1 a 1,24

M = definido por 7 atributos de proyecto y proceso que aumentan o disminuyen la estimación.

Ej: fiabilidad y complejidad del producto, etc.

Modelos empíricos de estimación

❖ COCOMO II – Modelo de reutilización

- » Se emplea para estimar el esfuerzo requerido al integrar código de reutilización o generado.
- » Para el código generado automáticamente, el modelo de persona/mes necesarias para integrar este código.

$$\text{PM}_{\text{auto}} = (\text{ASLOC} \times \text{AT}/100)/\text{ATPROD.}$$

Parámetros

PM = esfuerzo estimado en personas/mes.

AT = % de código adaptado que se genera automáticamente.

ATPROD = productividad de los ingenieros que integran el código.

ASLOC = nro de líneas de código en los componentes que deben ser adaptadas.

Modelos empíricos de estimación

❖ COCOMO II – Modelo de post-arquitectura

- » Se usa cuando está disponible un diseño arquitectónico inicial (se conoce la estructura). Entonces es posible hacer estimaciones para cada parte del sistema.
- » Las estimaciones están basadas en la misma fórmula básica $PM = A \times \text{Tamaño}^B \times M$ pero se utiliza un conjunto más extenso de atributos de M.
- » La estimación del número de líneas de código se calcula utilizando tres componentes:
 - **líneas nuevas** de código a desarrollar.
 - líneas de código fuente **equivalentes** (ESLOC), calculadas usando el nivel de reutilización.
 - líneas de código que **tienen que modificarse** debido a cambios en los requerimientos.
- » Estas estimaciones se añaden para obtener el tamaño del código (KLDC).

Modelos empíricos de estimación

❖ **COCOMO II** – Limitaciones de los modelos

- » Los modelos de COCOMO pueden ser complejos de usar, y requieren de datos históricos y experiencia para calibrar los factores de costo.
- » Se miden los costes del producto, de acuerdo a su tamaño y otras características, pero no la productividad.
- » La medición por líneas de código no es válida para orientación a objetos; entre otras cosas por la "reusabilidad" y la herencia, características de este paradigma (por ejemplo, puede implicar importante aumento en productividad, pero no en líneas de código).